Control flow

Control flow

1. `if`/`else` statements

   If condition is `TRUE`, then perform an action.

2. `while`

   Perform an action while the condition remains `TRUE`.

3. `repeat`

   Repeat an action forever.

We're going to look at `if`/`else`, then briefly at `for` loops.

`if` / `else`

```
if (CONDITION is TRUE) {
   DO SOMETHING
} else {
   DO SOMETHING ELSE
}
```

`if` / `else`

```
if (CONDITION is TRUE) {
   DO SOMETHING
} else {
   DO SOMETHING ELSE
}
```

```r
hour <- as.numeric(format(Sys.time(), "%H"))
if (hour < 12) {
  print("Good morning")
} else {
  print("Good afternoon")
}
```

```r
if (score >= 50) {
    grade <- "pass"
} else {
    grade <- "fail"
}
```

```r
if (score >= 50) {
    grade <- "pass"
} else {
    grade <- "fail"
}
```

We can have multiple conditions:

```r
if (score >= 70) {
    grade <- "A"
} else if (score >= 60) {
    grade <- "B"
} else if (score >= 50) {
    grade <- "C"
} else {
    grade <- "fail"
}
```

`if` checks a single value

`if` expects a condition of length 1. Give it a vector and you get an error:

```
> scores <- c(45, 72, 58)
> if (scores >= 50) "pass" else "fail"
Error in if (scores >= 50) "pass" else "fail" :
  the condition has length > 1
```

`if` checks a single value

`if` expects a condition of length 1. Give it a vector and you get an error:

```
> scores <- c(45, 72, 58)
> if (scores >= 50) "pass" else "fail"
Error in if (scores >= 50) "pass" else "fail" :
  the condition has length > 1
```

To check every element of a vector, use `ifelse()`:

```
ifelse(CONDITION, VALUE IF TRUE, VALUE
IF FALSE)
```

```
> ifelse(scores >= 50, "pass", "fail")
[1] "fail" "pass" "pass"
```

But don't overuse `else if` — it quickly becomes unreadable.

We'll introduce `if_else` and `case_when` in Session 2. These are a better option if you have multiple conditions to evaluate.

For example:

```
case_when(
  score >= 70 ~ "A",
  score >= 60 ~ "B",
  score >= 50 ~ "C",
  .default = "Fail"
)
```

Loops

`for` loops

A loop goes through a sequence of things and does something with each one.

```
for (THING in SEQUENCE) {
  DO SOMETHING
}
```

```
> for (i in 1:3) {
+   print(i * 10)
+ }
[1] 10
[1] 20
[1] 30
```

`i` is the current element of the sequence. You can call it anything:

```
for (m in month.name) {
  print(m)
}
```

Storing the results

To keep the results, create an empty vector first and fill it in:

```
> results <- numeric(5)
> for (i in 1:5) {
+   results[i] <- i^2
+ }
> results
[1]  1  4  9 16 25
```

Storing the results

To keep the results, create an empty vector first and fill it in:

```
> results <- numeric(5)
> for (i in 1:5) {
+   results[i] <- i^2
+ }
> results
[1]  1  4  9 16 25
```

But R is vectorised, so this does the same in one line:

```
> (1:5)^2
[1]  1  4  9 16 25
```

We rarely need loops in R.

- Use vectorised functions: `mean()`, `sqrt()`, `paste()`, `ifelse()`...
- To change a column in a data frame, use `mutate()` (Session 2).
- To repeat a task many times (e.g. import a folder of files), use `map()` from `purrr` (Session 5).

It is very rare to need a loop over the rows of your data.

But it's worth knowing what a loop looks like: you will see them in other people's code.